WHAT IS HYDRA?

Hydra is a configuration framework for Python applications

Hydra lets you compose configuration from multiple YAML files, override any value from the command line, and automatically log every run — without changing your code.

A config composer

Merges multiple YAML files into one config object using a structured defaults list. Your config is built from reusable parts, not one monolithic file.

A CLI generator

Every key in your config is automatically overridable from the command line with dot notation. No argparse. No Click decorators. Zero boilerplate.

A run manager

Every execution gets a timestamped output directory with your full config and logs saved automatically. Reproducibility is built in, not bolted on.

Open source · Facebook AI Research · hydra.cc · Powers Meta's fairseq, detectron2, and hundreds of research repos

THE PROBLEM

ML config management is broken

Most ML projects evolve through the same painful stages. Each fix creates new problems.

WITHOUT Hydra

❌ Hardcoded values

Parameters live in source code. Every experiment requires an edit, a save, and a rerun—and leaves no trace of what changed.

❌ CLI flags don't scale

argparse/Click start clean but balloon: 15-parameter signatures, if/elif dispatch chains, flags that leak across models.

❌ Experiments vanish

No automatic record of what you ran. That 0.92 F1 score? Gone unless you remembered to log it. Reproducibility is manual heroism.

❌ Adding params is costly

One new hyperparameter means editing the decorator, the signature, and the dispatch logic—in three separate places.

WITH Hydra

✅ Config lives in YAML

Parameters are declared once in structured files. Code stays clean. Every value is visible, versionable, and diff-able.

✅ One flag, whole config

model=random_forest swaps the entire model config. No if/elif. No coupled parameters. No pollution between models.

✅ Every run is logged

Hydra creates a timestamped output directory per run. Config, overrides, and logs are saved automatically. Nothing disappears.

✅ +param=value, done

Add any parameter on the fly from the CLI with the + prefix. No code change. No redeploy. Immediate experimentation.

cfg is not a dict — it's a DictConfig

When Hydra calls your main(cfg), it passes an OmegaConf DictConfig — a structured, smart config object, not a plain Python dictionary.

Plain dict ❌

```
# You'd have to write this yourself:
cfg = {
    "model": {
        "n_estimators": 100,
        "max_depth": None
    },
    "random_state": 42
}

# Access is verbose:
cfg["model"]["n_estimators"]

# No interpolation, no merging,
# no type safety, no validation
```

OmegaConf DictConfig ✅

```
# Hydra gives you this automatically:
# cfg is a DictConfig

# Dot-notation access:
cfg.model.n_estimators   # 100
cfg.random_state          # 42

# Bracket access also works:
cfg["model"]["n_estimators"]

# In(ex)istent key → clean error:
cfg.model.typo  # raises MissingMandatoryValue
```

💡 `from omegaconf import OmegaConf, DictConfig` — OmegaConf is the library; DictConfig is the type hint for cfg

What DictConfig gives you

Read-only by default

Inside @hydra.main, cfg is frozen. Accidental mutations raise an error — your config stays authoritative.

```
cfg.model.n_estimators = 999
# → raises HydraException

# To mutate: use OmegaConf.to_container()
d = OmegaConf.to_container(cfg, resolve=True)
# d is now a plain mutable dict
```

Type coercion

Values are coerced to the type from your YAML. Strings that look like ints become ints. null becomes None.

```
# In YAML:
#   n_estimators: 100
#   max_depth: null

type(cfg.model.n_estimators)  # <class 'int'>
cfg.model.max_depth           # None
```

Interpolation is live

When you read a ${} reference, OmegaConf resolves it at access time — not at load time. The value is always current.

```
# In YAML:
#   random_state: 42
#   model:
#     seed: ${random_state}

cfg.model.seed   # resolves to 42
```

Structured access patterns

DictConfig supports iteration, membership testing, and conversion — all the dict patterns you already know.

```
# Iterate keys:
for key in cfg.model:
    print(key, cfg.model[key])

# Membership test:
"n_estimators" in cfg.model  # True
```

Anatomy of @hydra.main

The decorator is the entry point for everything Hydra does. Three parameters control how it starts up.

The decorator

```
@hydra.main(
    config_path="../conf",   # ①
    config_name="config",   # ②
    version_base="1.3",     # ③
)
def main(cfg: DictConfig):
    # cfg is fully composed here
    # logging is configured
    # output dir exists
    # CWD has been changed
    ...
```

① config_path

str | None

Path to the directory containing your YAML files, relative to the calling Python file. Use None if configs are embedded as Structured Configs.

② config_name

str | None

The filename (without .yaml) of the top-level config to load. This is the root of your defaults list composition.

③ version_base

str | None

Controls which Hydra version's defaults to use. Always set this explicitly — omitting it raises a warning. Use "1.3" for current Hydra behavior.

```
# Minimal working example (no config file needed for testing):
@hydra.main(config_path=None, config_name=None, version_base="1.3")
```

@HYDRA.MAIN

What @hydra.main Does at Startup

Before your first line of code executes, Hydra does a lot of setup. Understanding the sequence prevents surprises.

1

Config composition

Hydra loads config_name.yaml, resolves its defaults list (loading group configs), merges them in order, then applies CLI overrides. You receive the final merged DictConfig.

2

Output directory created

A timestamped outputs/ subdirectory is created per hydra.run.dir. The .hydra/ folder inside it gets config.yaml, overrides.yaml, and hydra.yaml written immediately.

3

Working directory changed

Hydra calls os.chdir() into the new output directory. Relative file paths in your code now point there — not your project root. Use HydraConfig.get().runtime.cwd or the original_cwd to get back.

4

Logging configured

Python's logging system is wired up using Hydra's logging config. All log.info() calls are captured to the run's .log file automatically — zero setup needed in your code.

⚠ Step 3 (CWD change) is the most common source of confusion. If open('data.csv') worked before adding Hydra and now fails, this is why.